Outlook

[PLDI'26 SRC] Accepted submission #45 "K-Sentry: A Verified Order-Sensitive..."

From PLDI'26 SRC HotCRP <noreply-pldi26src@hotcrp.com>
Date Fri 4/24/2026 10:29 AM
To Ghimire, Sudrit <aui25@txstate.edu>

Dear author,

The PLDI 2026 Student Research Competition (PLDI'26 SRC) program committee is delighted to inform you that your submission #45 has been accepted to appear at the conference poster session. Congratulations!

* Title: K-Sentry: A Verified Order-Sensitive Telemetry Accumulator
* Author: Sudrit Ghimire (Texas State University)
* Site: https://nam04.safelinks.protection.outlook.com/?url=https%3A%2F%2Fpldi26src.hotcrp.com%2Fpaper%2F45%3F%3D&data=05%7C02%7Caui25%40txstate.edu%7Ccaaac70fd7d4484036e308dea2163c97%7Cb19c134a14c94d4caf65c420f94c8cbb%7C0%7C0%7C639126413578468904%7CUnknown%7CTWFpbGZsb3d8eyJFbXB0eU1hcGkiOnRydWUsIlYiOiIwLjAuMDAwMCIsIlAiOiJXaW4zMiIsIkFOIjoiTWFpbCIsIldUIjoyfQ%3D%3D%7C0%7C%7C%7C&sdata=qaG6ljqNAr1e5HZY2frS1f3%2Bve%2FdQYepkO%2FDtYA1p3U%3D&reserved=0

Our expectation is that you update your extended abstract to incorporate feedback from the reviews you received. We will assign a member of the program committee (a "shepherd") who can help you with this process if you have any questions. The deadline to submit the updated camera-ready PDF is *May 15, 2026*.

We hope to see you at PLDI,
Gowtham & Alexa

Review #45A
==========

Overall merit
---
3. Accept

Reviewer expertise
---
2. Some familiarity

Technical Feedback

This paper describes a log accumulator that is sensitive to modifications in the log, ensuring that if a log is consistent with its checksum, then the log was unlikely to be tampered with. The paper is reasonably on topic for PLDI but might be more at home in a crypto or systems venue.

The origin of the accumulator is not explained until too late. Why is x_i q^i^2 mod p the right accumulator? Put another way, I can see that it works, but I wondered why i^2 is the right thing. I see this is explained later in L233, but I don't quite understand the tradeoffs quantitatively. Also, I'm not quite sure how resilient this mechanism is to various kinds of attacks: the paper discusses the probability of collisions at random (if I understood correctly), but I'm unclear how easy it would be for an attacker to modify the log in a way that still collides with the original.

The paper says that the code was verified with Verus, but lacking experience in Verus, I would have appreciated a hint as to what these theorems look like in Verus and what kind of proof was required. There's a summary around L200, but I'm curious what the actual code looks like.

The paper does not put the performance measurement in context. There are several other mechanisms summarized in Table 1; a performance comparison with those would be very helpful. For example, if this takes hardly any more computation than XOR, then it's clearly very cheap.

I wonder about the benefits of verification. Were any bugs found? Is there a reason to believe that the unverified algorithms are likely buggy? The fact that it was verified is viewed as an advantage (Table 1), so is there a reason we should use K-Sentry rather than verifying an existing algorithm (e.g. is verifying the other implementations infeasible)? Or is the main advantage of K-Sentry not that it is verified, but instead that it is order-sensitive and more compact (L224)?

Writing Feedback

I think the paper could do a better job of positioning. Why do we need a new log integrity mechanism with respect to practical needs? That is: the paper presents the technical shortcomings of prior approaches, but I'm unclear on whether these shortcomings present practical problems.

L173: I found the || operator unclear until I read the "why it matters" paragraph. Moving that paragraph earlier would help clarify, as would explaining the || operator explicitly. In general, I think putting the "why it matters" paragraph before the theorem statements might help.

* *

Review #45B
==========

Overall merit
---
2. Unsure

Reviewer expertise
---

2. Some familiarity

Technical Feedback

Summary
The paper proposes a novel telemetry algorithm over a finite field and describes a formally verified implementation of it.

Comments
At a high level I think this paper focuses too much on describing and justifying what seem to be fairly trivial theorems, and not enough time justifying the choice of hashing function. Does the quadratic weighting scheme it have particular properties that make it stronger than a traditional rolling hash? Is it somehow more amenable to verification? You briefly mention this in a single sentence in section 5, but I think these are more interesting questions worth spending time on; you definitely have the space for it.

You mention "verification overhead" as a reason your technique has lower throughput compared to standard rolling hashes, but surely the verification isn't happening at run-time, right?

Writing Feedback

Context
I think a formally verified implementation of something important like a hash function is reasonably well-motivated for a PL audience.

Technical presentation
As I've mentioned above, I think the paper spends too much time overexplaining the theorems while still missing important details. For example, what are the `nonzero_poly_has_few_roots` and `ramanujan_seq1/2/3` lemmas mentioned in section 3? These are named but as far as I can tell never defined.

I think the theorems presented in 3.1-3.4 are fairly obvious. Its definitely good to state them, but you don't need to spend so much time justifying why these are the theorems you chose to prove. In particular, I think the "_why it matters_" sections are unecessary and often kind of confusing; in particular, these read kind of LLM-generated to me.

Evaluation
Seeing throughput numbers is good, and I appreciate comparing to traditional rolling hash techniques (though again, I'm still curious why there should be any verification overhead). Along those lines, if a core contribution claim being made here is "this accumulator has better properties with respect to localizing errors", I'd expect to see some experiments showing its ability to detect multiple errors at once, etc.

* *

Review #45C
==========

Overall merit
---
3. Accept

Reviewer expertise
---
1. No familiarity

Technical Feedback

This paper presents K-Sentry which is a telemetry integrity mechanism. The goal is to have a mechanism that is streaming, order-sensitive and verified. The paper contrasts K-sentry with other telemetry integrity mechanisms. It explains the mathematical formula for K-sentry, and the key theorems involved in its verification. It concludes with an evaluation of the prototype.

Question: why does K-Sentry need to have a quadratic formula specifically vs a linear formula or a cubic formula? I assume there is a number theoretic reason, but it would be nice to know specifics. Also, though I am convinced the square gives you a good enough probability bound and other properties you need, would anything be improved with a higher power like a cube?

Writing Feedback

This paper would have benefitted from a longer introduction that explained the other telemetry approaches that K-sentry was being compared to more extensively (with citations where one could read more about other approaches would be helpful). For example, the table shows Merkle trees, but Merkle trees are not mentioned in the introduction. Also, when discussing the mathematical formula in the introduction, what are x, n, p and q? These variables are later defined in Section 2, but it is confusing in Section 1. The example in Section 2 was useful, as was the visualization and intuition behind the theorems. The evaluation was well explained.

* *

Review #45D
==========

Overall merit
---
2. Unsure

Reviewer expertise
---
3. Knowledgeable

Technical Feedback

The paper considers the problem of assigning a unique hash value to a sequence of values, for which it

defines a K-sentry accumulator which can computed in a streaming fashion and which also ensures that the hash value is sensitive to the order of values in the sequence. In addition, the implementation in Rust is also verified to satisfy basic functional properties.

While the paper seems like a good exercise in verification of a useful functionality which satisfies some nice properties and is also quite efficient, I am unsure whether there are any non-trivial research challenges involved in the process. The definition of the K-sentry accumulator is quite straightforward (basically a weighted average of values where the weights are derived by repeatedly squaring a base q), and it implementation as shown in Fig. 1 is very simple. The properties that are verified are also quite trivial, basically just stating that every step of the implementation is correct. As such, I do not see any specific verification challenges; this type of problem would be covered in a standard graduate course on program verification. The implementation can actually be improved a bit: in every iteration, it seems that the weight q^{i^2} is computed from scratch, but instead a running value can be maintained appropriately updated in each iteration to take advantage of the weights computed in the previous iterations.

It also seems to me this method of computing weights could quite quickly result in very large weight values for the later indices, which could result in very large hash values. Is there any specific reason why this particular method of computing weights was used?

Overall, I feel that while this is a good first step in verifying lightweight telemetry implementations, which hasn't been before, the paper is a bit lightweight in terms of contributions. There are other well-known telemetry implementations such as Rolling hash and Merkle tree which seem to satisfy the same properties as the proposed K-sentry. Maybe the authors can consider verification of these well-known techniques as well.

Writing Feedback

The writing is mostly OK. Some minorn comments:
- What is K in the word 'K-Sentry' refer to? I don't see any such notation being used anywhere in the paper.
- In the abstract and the introduction, symbols such as q and p are used without being defined, which comes later in Section 2. Please add the meanings of symbols before using them.

CAUTION:. This email originated from outside the TXST network. Do not click links or download files unless you know the sender and content are safe.

Outlook

[Sigmetrics '26 SRC] Submission #10 "ksentry: A Verified Order-Sensitive..."

From Sigmetrics '26 SRC HotCRP <noreply-sigmetrics26src@hotcrp.com>
Date Wed 4/1/2026 10:52 AM
To Ghimire, Sudrit <aui25@txstate.edu>

Dear author,

The ACM Sigmetrics 2026 Student Research Competition (SRC) (Sigmetrics 2025 SRC) program committee is regret to inform you that your submission #10 has been rejected and will not participate in the SRC.
At same time, we still appreciate the interesting ideas and potential in your work and would like to invite you to present your poster in the student poster session.

* Title: ksentry: A Verified Order-Sensitive Telemetry Accumulator
* Site: https://nam04.safelinks.protection.outlook.com/?url=https%3A%2F%2Fsigmetrics26src.hotcrp.com%2Fpaper%2F2310&data=05%7C02%7Caui25%40txstate.edu%7Ce9b9c43bbb6f4d808fdf08de9006b1a9%7Cb19c134a14c94d4caf65c420f94c8cbb%7C0%7C0%7C639106555611064634%7CUnknown%7CTWFpbGZsb3d8eyJFbXB0eU1hcGkiOnRydWUsIlYiOiIwLjAuMDAwMCIsIlAiOiJXaW4zMiIsIkFOIjoiTWFpbCIsIldUIjoyfQ%3D%3D%7C0%7C%7C%7C&sdata=oGDoynSNRt%2FEcaA7KNRMsxLOlQs18VrsNf5%2B4gDg3F8%3D&reserved=0

Visit the submission site for reviews, comments, and related information. Reviews and comments are also included below.

The next steps are:

1) If you decide to present your work at the poster session, please confirm by replying to this email.

2) Register for Sigmetrics. We are in the process of confirming the registration discount for student poster presenters; we will send out details about the registration soon.

3) Prepare a poster to be presented at the poster session during Sigmetrics 2026. We will send out more details closer to the actual conference.

More details can be found here:
https://nam04.safelinks.protection.outlook.com/?url=https%3A%2F%2Fwww.sigmetrics.org%2Fsigmetrics2026%2Fstudent_activities.html&data=05%7C02%7Caui25%40txstate.edu%7Ce9b9c43bbb6f4d808fdf08de9006b1a9%7Cb19c134a14c94d4caf65c420f94c

8cbb%7C0%7C0%7C639106555611085213%7CUnknown%7CTWFpbGZsb3d8eyJFbXB0eU1hcGkiOnRydWUsIlYiOiIwLjAuMDAwMCIsIlAiOiJXaW4zMiIsIkFOIjoiTWFpbCIsIldUIjoyfQ%3D%3D%7C0%7C%7C%7C&sdata=zNk%2BiUjkfCBMr8vPkIlxhms%2FC4lTDp7%2FfuUvk4SXpD8%3D&reserved=0

We look forward to seeing you at Sigmetrics for chatting more about your project.

Contact Emilio Incerto <emilio.incerto@imtlucca.it> or Shaolei Ren <shaolei@ucr.edu> with any questions or concerns.

Emilio Incerto & Shaolei Ren
ACM Sigmetrics 2026 SRC Co-chairs

Review #10B
==========

Overall merit
---
3. Weak accept

Reviewer expertise
---
1. No familiarity

Paper summary

Overall the paper presents a nice design, K-Sentry, a compact streaming telemetry accumulator. The author presented a formal bound and a Rust implementation.

Comments for authors

Thank you for submitting to SRC!
The paper has nice theoretical contribution.

Some suggestions:
1) Facts in papers need references to back up. For example, the first sentence in intro, "Modern cloud systems generate high-volume telemetry streams for monitoring and automated control." here you need to cite several papers/news articles to prove that your claim is true.
2) A brief related work paragraph is needed.
3) you have table 1 and table 2 presented but you did not link them in the text
4) Implementation details missing. Where did you implement? Rust? How? Evaluation and experimental set up? You don't need to write a long technical report, but some levels of details are still needed.

* *

Review #10C

==========
========

Overall merit
---
3. Weak accept

Reviewer expertise
---
1. No familiarity

Paper summary

The problem is practical and the approach is well-scoped. Table 1 makes a clear case for why K-Sentry sits in a useful spot between lightweight checksums. And the prototype results on real Google telemetry traces are convincing. The collision bound analysis is also a welcome addition.

* *

Review #10D
==========
========

Overall merit
---
3. Weak accept

Reviewer expertise
---
2. Some familiarity

Paper summary

This paper proposes K-Sentry, a constant-memory telemetry accumulator that binds each log value to its position using q^{i^2} mod p, with a Rust implementation mechanically verified in Verus. The paper states refinement properties for the streaming computation, gives a probabilistic collision bound for distinct equal-length traces, and reports prototype experiments on synthetic data and a Google cluster trace showing order-sensitive tamper detection at multi-million-log throughput.

Comments for authors

This is a clear and well-motivated paper, and the formal-verification angle is appealing. I especially liked the emphasis on compositional aggregation and the effort to connect mechanized refinement to a concrete telemetry application.

For improvements, my main suggestion is that the authors could discuss in more details the paper's positioning relative to existing order-sensitive fingerprinting/hash ideas. The paper should explain more

clearly why the quadratic weighting is preferable to more conventional ones. Relatedly, the claims about exact mismatch localization feel somewhat stronger than what is formally established in the current version; it is not fully clear whether localization is a theorem, an experimental observation, or something that requires extra prefix/segment information beyond a single final accumulator value.

Moreover, the evaluation is promising but still fairly light. The paper would be stronger with a broader comparison set, more detail on the cost of modular exponentiation and parameter selection, and a cleaner separation between accumulator overhead and surrounding pipeline overhead.

* *

Review #10E
==========
==========

Overall merit
---
2. Weak reject

Reviewer expertise
---
3. Knowledgeable

Paper summary

This paper presents K-Sentry, an order-sensitive streaming accumulator for telemetry traces, defined as \(A(w;q,p)=\sum_i x_i q^{i^2} \bmod p\). The paper argues that binding each log entry to a position-dependent quadratic weight enables detection of edits, swaps, insertions, deletions, and truncation while retaining constant-memory streaming execution. A key contribution is a mechanized Verus refinement argument for the Rust implementation, covering weight correctness, step correctness, prefix correctness, and compositional aggregation. The paper also states a collision bound of at most \((n-1)^2/p\) for distinct equal-length traces under uniformly random \(q\), and reports prototype throughput of \(3.41\times 10^6\) logs/s on synthetic data and \(4.05\times 10^6\) logs/s on a public Google cluster trace.

Comments for authors

This is a clear, concise, and easy-to-follow SRC submission. The problem is practically relevant, since order-sensitive integrity in telemetry pipelines matters for diagnosis and control, and the paper does a good job motivating why simple checksums or XOR summaries are insufficient. The strongest part of the work is the formal-methods angle: connecting a mathematical accumulator specification to executable Rust through Verus is a meaningful systems contribution, especially in a short student-research format. I also found the compositional aggregation property well motivated for distributed telemetry settings. The paper is readable, focused, and better structured than many submissions of this length.

My main concern is that the paper currently overstates its integrity and tamper-detection story relative to the formal guarantee it actually establishes. As written, the core result is a randomized collision bound

for a fingerprint-like construction under uniformly chosen \(q\). That is not the same thing as a cryptographic integrity guarantee against an adaptive adversary. The paper repeatedly uses language such as 鈥淪ttacks detected,鈥 鈥淭amper detection,鈥 and 鈥淔orensics/compliance,鈥 but it does not specify whether \(q\) is secret, how it is chosen, whether it is reused across streams, or what the attacker is assumed to know. Without an explicit adversarial model, the construction reads much more like an order-sensitive polynomial fingerprint than an authenticator. I think this distinction matters a lot, and the claims should be sharpened accordingly.

My second major concern is that the algorithmic design choice is under-motivated. The paper uses quadratic exponents \(q^{i^2}\), but does not explain why this is preferable to the standard linear polynomial fingerprint \(\sum_i x_i q^i\), which is already order-sensitive and, at least at first glance, seems to admit a tighter collision bound of order \((n-1)/p\) rather than \((n-1)^2/p\). If the quadratic indexing is essential, the paper needs to explain what property it enables that a linear construction does not. Right now, this choice feels somewhat arbitrary, which weakens the novelty argument.

A related issue is the paper鈥檚 repeated claim of 鈥渆xact mismatch localization.鈥 The formal refinement statements are about weight, step, prefix, and compositional correctness, while the experiments report localization behavior such as a single-bit tamper at index 9 and a swap localized to boundary indices \(\{0,18\}\). However, the paper never explains what localization algorithm is actually used, what extra information it requires, what its complexity is, or whether localization is part of the verified story or merely an empirical behavior of the prototype. From the final accumulator state alone, one can test equality, but not obviously recover the location of a mismatch. If localization requires checkpoints or intermediate states, that should be stated explicitly and reconciled with the constant memory narrative. As currently written, the localization claim is too strong for the amount of technical detail provided.

I also think the verification boundary should be described more carefully. The paper does acknowledge that the root-counting lemma used for the collision bound is outside the mechanized proof, which is good, but this weakens the verified headline somewhat. For an SRC paper this is not fatal, yet it does mean the submission should be more precise about what is fully verified and what remains trusted mathematical reasoning.

The evaluation is also fairly thin for the breadth of the claims. The correctness examples are tiny (\(n=19\)), the real-trace experiment uses 100,000 entries, and the performance comparison is mostly against a rolling-hash baseline that is about 55x faster. The paper says this is a tradeoff for stronger guarantees, but it does not really show when that tradeoff is operationally attractive. I would have liked to see a more systematic scaling study, a breakdown of where the overhead comes from, and a clearer discussion of how acceptable the reported collision probability is when many streams or windows are checked in production. The reported 10.8 GB memory usage is also attributed to pipeline overhead without enough detail to interpret it.

Finally, the related-work positioning is too sparse. The paper should engage more directly with polynomial fingerprinting and rolling-hash literature, authenticated streaming mechanisms, and verified implementations of integrity-related primitives or authenticated data structures. My sense is that the mechanization angle may be the most novel part of the work, but the current draft does not frame that contribution as clearly as it could.

Overall, I think this is a promising student project with a nice verified-systems flavor, but the current version needs a much clearer threat model, a more convincing justification for the quadratic

construction, a precise explanation of localization, and a deeper evaluation. My recommendation is weak reject in its current form, though I do think the project could become a solid SRC presentation with a more careful and less overstated writeup.

CAUTION:. This email originated from outside the TXST network. Do not click links or download files unless you know the sender and content are safe.

Gmail Sudrit Ghimire <sudritghimire@gmail.com>

FMCAD 2026 Student Forum: Notification for paper 228

1 message

FMCAD 2026 Student Forum <fmcad2026studentforum@easychair.org> 22 July 2026 at 06:43
To: Sudrit Ghimire <sudritghimire@gmail.com>

Dear Sudrit Ghimire,

On behalf of the FMCAD 2026 Student Forum Program Committee, we are happy to inform you that the following submission has been accepted for presentation:

TimeFence: Practical Sequence Evidence for Linux Telemetry

Congratulations! All papers underwent a thorough peer review process with three reviews each. These reviews and Program Committee discussions were carefully considered in the final decisions.

Please find the full text of the reviews for your submission below. We hope that the provided feedback will help improve your work and prepare the presentation for the Student Forum.

We will be in touch shortly with more details regarding the format of the presentation and poster.

Please note that all student forum participants are required to register for the main FMCAD conference. Early registration with reduced pricing ends on August 1.
Please reach out to us if you have any questions.

Thank you for submitting to the FMCAD 2026 Student Forum. We look forward to seeing you at TU Graz!

Best Regards,
Tanja Schindler and Lee Barnett
FMCAD 2026 Student Forum Co-Chairs

SUBMISSION: 228
TITLE: TimeFence: Practical Sequence Evidence for Linux Telemetry

---------- REVIEW 1 ----------

SUBMISSION: 228
TITLE: TimeFence: Practical Sequence Evidence for Linux Telemetry

---------- Overall evaluation ----------
SCORE: 1 (weak accept)
----- TEXT:
Summary
The paper introduces TimeFence, which uses “K-Sentry”, a verified finite-field accumulator, to compute order-sensitive digests of telemetry data. The technique is used for determining whether telemetry data (after it leaves the collection path) has been manipulated in any way (e.g. events being dropped, processed out-of-order, new events being inserted etc.), which can impact the use of the data for incident response and auditing purposes. The arithmetic invariants of the accumulator are verified in Verus. The overhead of the system is evaluated on 5 different workloads.

Evaluation
The problem is clearly stated, is well-motivated, and distinguished from existing tools, such as Falco and auditd. The verification boundary section is great in that it describes the scope of what Verus covers and doesn’t cover along with the limitations of K-Sentry. The preliminary evaluation shows promising results of TimeFence when compared to Falco and auditd.

The bulk of the paper focuses on the “systems” aspects and less so on the formal aspects beyond stating that the accumulator core is verified in Verus. For example, I would have liked to see more details on soundness against tampering of events. I’d encourage the authors to focus more on the “verified” aspects of the approach to make the FMCAD fit more clear.

Comments:
* FMCAD student forum allows trimming the related works section to what's crucial, but the paper does not include any. It would be nice with a least a couple of references to get more context on the contribution
* There are no references in the paper, e.g. auditd, Falco, and Verus. Can you add citations?
* In table 1: What does "Checkpoint localization" and "19/19 at interval 1000" mean?
* The abstract mentions that the range of the overhead is 0.83 - 6.67% but in table 2, the "exec" workload has -0.31%. Should one of them be corrected? What's the intuition behind a negative overhead?

---------- REVIEW 2 ----------

SUBMISSION: 228
TITLE: TimeFence: Practical Sequence Evidence for Linux Telemetry

---------- Overall evaluation ----------
SCORE: 0 (borderline paper)
----- TEXT:
Summary

The paper is about TimeFence, a layer for Linux telemetry. The goal is to get a streaming, compact, order-sensitive digest that can be checked later against evidence certificates. To this end, a verified finite field accumulator is employed. Several experiments are presented.

Evaluation

Overall, the structure of the paper is clear and the work seems relevant, but the presentation could be improved in the following points.

* The extended abstract is very hard to read because it is basically assumed that the reader knows the approach. Core terminology needs to be introduced.

* There are a lot of formulas, which are not well explained. All the symbols that occur in the formulas need to be introduced.

* The status of the approach is not clear. Is there already some tool/paper published?

* Table 1 does not seem to be referenced in the text - some integration would be beneficial.

* It is hard to assess what we can learn from the experiments.

* No references are given. Some other tools are mentioned, but the state of the art that is improved by this research should be made more explicit.

* Finally, the overall research question addressed in this work need some more elaboration.

---------- REVIEW 3 ----------

SUBMISSION: 228
TITLE: TimeFence: Practical Sequence Evidence for Linux Telemetry

---------- Overall evaluation ----------
SCORE: 2 (accept)
----- TEXT:
The paper presents an approach for computing a hash-like summary of a log of events that is sensitive to various classes of modifications, including changing the order of events. This approach is evaluated in the context of Linux kernel telemetry.

Overall, the paper presents an interesting approach with solid experimental results. The implementation is also verified in Verus.

---------- FEEDBACK ----------
There is no author feedback for this paper.

TEXAS STATE UNIVERSITY
Honors College

October 7, 2026

Sudrit Ghimire
aui25@txstate.edu

Mentor: Dr. Hongchi Shi
hs15@txstate.edu

Dear Sudrit,

Congratulations! You have been selected to receive a $1000 **Undergraduate Research Fellowship (URF) award** to support your project, "Constructing and Verifying Widely Digitally Delicate Primes." We're excited to support your research and look forward to seeing what you accomplish! Congratulations and thanks also to your mentor, Dr. Hongchi Shi, copied on this email.

As part of your fellowship, the Undergraduate Research team expects you to work with your mentor to present or showcase your work before graduation at the Undergraduate Research Conference (URC) or another appropriate forum at Texas State. Keep us updated on your presentation plans and let us know if they or your graduation plans change. We're happy to help you find a suitable venue if needed, so don't hesitate to reach out. We also will follow up if we don't hear from you.

We'd also love to celebrate your achievement by recognizing you on the Undergraduate Research website and through other university communications. If you're comfortable with us publicizing your award, please **reply with your permission and a high-quality headshot.**

When presenting or publishing work supported by your award, please acknowledge the **Texas State Undergraduate Research Fellowships** by name. We also hope you'll stay in touch after graduation and share where your research and interests take you!

A separate email with guidelines for accessing and using your fellowship funds will be sent soon. In the meantime, please complete the form below so we can begin setting up your fellowship account:

Fall 26 URF Award – Administrative Information

Congratulations again on this exciting accomplishment! We wish you every success as you continue your research journey.

Warm regards,

Undergraduate Research Team
Honors College | Texas State University

Outlook

[WATCH 2026] Submission #36 "TraceLock: Verified Streaming Evidence..."

From WATCH 2026 HotCRP <noreply-watch2026@hotcrp.com>
Date Fri 9/4/2026 3:02 PM
To Ghimire, Sudrit <aui25@txstate.edu>

Dear Sudrit Ghimire,

Congratulations! We are pleased to inform you that your paper has been accepted for presentation at the workshop.

Please refer to the enclosed reviews and consider the reviewers' comments while preparing the camera-ready copy. The camera-ready submission deadline is September 11, 2026. Detailed camera-ready submission instructions will be provided by ACM at a later date.

Congratulations again, and we look forward to seeing your work at the workshop!
* Title: TraceLock: Verified Streaming Evidence for Linux Telemetry
* Site: https://nam04.safelinks.protection.outlook.com/?url=https%3A%2F%2Fwatch2026.hotcrp.com%2Fpaper%2F36&data=05%7C02%7Caui25%40txstate.edu%7Cf8cdd8a7575b444886bd08df0abf7e18%7Cb19c134a14c94d4caf65c420f94c8cbb%7C0%7C0%7C639241489724981695%7CUnknown%7CTWFpbGZsb3d8eyJFbXB0eU1hcGkiOnRydWUsIlYiOiIwLjAuMDAwMCIsIlAiOiJXaW4zMiIsIkFOIjoiTWFpbCIsIldUIjoyfQ%3D%3D%7C0%7C%7C%7C&sdata=RVblSfX3nLy89zhquYOYKG4hHf%2BSGm5ibReHR%2BwwAxY%3D&reserved=0

Contact WATCH.miniconf@gmail.com with any questions or concerns.

- WATCH 2026 Submissions
https://nam04.safelinks.protection.outlook.com/?url=https%3A%2F%2Fwatch2026.hotcrp.com%2F&data=05%7C02%7Caui25%40txstate.edu%7Cf8cdd8a7575b444886bd08df0abf7e18%7Cb19c134a14c94d4caf65c420f94c8cbb%7C0%7C0%7C639241489725005731%7CUnknown%7CTWFpbGZsb3d8eyJFbXB0eU1hcGkiOnRydWUsIlYiOiIwLjAuMDAwMCIsIlAiOiJXaW4zMiIsIkFOIjoiTWFpbCIsIldUIjoyfQ%3D%3D%7C0%7C%7C%7C&sdata=lqpFvljwjEnBbZbE1JcV7rR6mdgWJUkykN%2FKvpOhrfw%3D&reserved=0

Review #36A
==========

Overall merit
---
4. Accept

Reviewer expertise
---
3. Knowledgeable

Paper summary

The paper proposes TraceLock, built around K-Sentry, for Linux telemetry, evaluated its implementation in Rust, and presented and compared results.

Comments for authors

The paper is timely, and discusses a standing issue that has been there for a while.
The presentation is good, and the readability is good.
Stable observed rates, workload overhead and resource usage are all impressive.
Limitations of the implementation and evaluation are discussed.
Are there scalability concerns or reliability issues?

* *

Review #36B
==========

Overall merit
---
3. Weak accept

Reviewer expertise
---
3. Knowledgeable

Paper summary

The manuscript introduces **TraceLock**, a verified, order-sensitive streaming evidence layer designed to guarantee the integrity of Linux telemetry event sequences post-collection. Telemetry sequences are highly susceptible to malicious post-collection tampering, including event omission, insertion, reordering, or same-length replacement—attacks that standard length checks or unordered set summaries fail to detect.

The core of TraceLock is **K-Sentry**, a finite-field accumulator that maps canonicalized system events into field elements and computes order-sensitive checkpoints using non-affine, triangular positional weights ($T_i = \frac{i(i+1)}{2}$). The non-affine exponent design prevents the algebraic shift-rebasing weakness inherent in standard affine rolling hashes, making it impossible for an attacker to rebase shifted segment digests using a single scalar independent of the segment's contents. The mathematical core is formally verified in **Verus** to ensure that the Rust-implemented accumulator conforms strictly to its specification. The authors implement a live eBPF-based collector in Rust tracing `execve`, `openat`,

and `connect` events, and evaluate the prototype on native AWS EC2 workloads, demonstrating low overhead (0.83% to 6.67%) compared to heavier runtime detectors like Falco and auditd.

Comments for authors
---
1. Misleading "Apples-to-Oranges" Baseline Comparison in Table 10
In Table 10 ("Accumulator-cost ablation at 100k events"), the authors compare the throughput of K-Sentry (665,247 events/s) with a Python-based Merkle tree baseline. The table claims the Merkle baseline achieves a massive throughput of **10,233,681 events/s** based on a runtime of **9.77 ms** for 100,000 events.
* As noted in the text, the Merkle baseline is "chunk-level" with an interval of 1,000. This means the Merkle code is only performing **100 cryptographic hashes** (one per chunk) rather than processing 100,000 events individually.
* Presenting this as "10.2 million events/s" is highly misleading. A chunk-level tree does not provide per-event fine-grained updates or order-sensitivity within the chunk without additional local storage. Comparing a chunked hash to a per-event mathematical accumulator, and then plotting them on the same "Events/s" metric, is an unscientific comparison. The authors must either run a true per-event Merkle baseline or explicitly adjust the throughput metric to reflect actual hashed blocks.

2. Physically Impossible Negative Overhead in Table 6
In the performance evaluation (Table 6), the authors list the system overhead for the `process_exec` workload under TraceLock as **`-0.31%`**.
* A software monitoring layer—which attaches eBPF kprobes, transfers data through a ring buffer, canonicalizes text strings, maps hashes to a finite field, and executes mathematical updates—cannot have a negative performance cost.
* While negative overheads can occasionally occur in systems benchmarking due to CPU cache alignments, scheduler noise, or thread-scheduling jitter, reporting a negative overhead without any explanation suggests a lack of calibration or statistical validation. The authors should explain this measurement noise or report the confidence intervals (standard deviation) over multiple runs to show that the negative value is within the margin of error.

* *

Review #36C
==========

Overall merit
---
4. Accept

Reviewer expertise
---
1. No familiarity

Paper summary

The paper introduces TraceLock, a verified order-sensitive streaming evidence layer for Linux telemetry designed to prove that an ordered telemetry sequence has been preserved after collection. TraceLock is based on a finite-field accumulator, K-Sentry, using non-affine exponents in order to prevent digest-only scalar rebasing independent of segment contents.

Comments for authors

The paper is interesting, and clearly outlines the theoretical framework, although in some places it seems a bit too concise to me, especially regarding the implementation details; it would probably be helpful to include a reference to the code used.

CAUTION:. This email originated from outside the TXST network. Do not click links or download files unless you know the sender and content are safe.